Machine Learning Project Documentation

Adaptive Digital Wellbeing Coach

Model Refinement and Test Submission

Group 8

Model Refinement

1. Overview

This phase extends our earlier model exploration by training **Logistic Regression, Random Forest** and **XGBoost** on *dataset_v1*. We compare them with a *fixed-rule baseline* and examine whether a small hyperparameter search and validation-based decision thresholds improve the balance between precision and recall. The results concern a behavioural proxy for elevated phone use. They do not establish that the model identifies use that a person considers unwanted.

2. Model Evaluation

Our previous experiments used a separate raw-log processing adapter and compared the rule baseline, Logistic Regression and Random Forest. They showed a *trade-off*: **Logistic Regression** made fewer positive predictions and missed more positive cases, while **Random Forest** detected more cases but produced more false positives. The previous results and confusion/ROC/PR figures are recorded in *reports/model_exploration_all_342/* and *reports/model_exploration_thresholds/*.

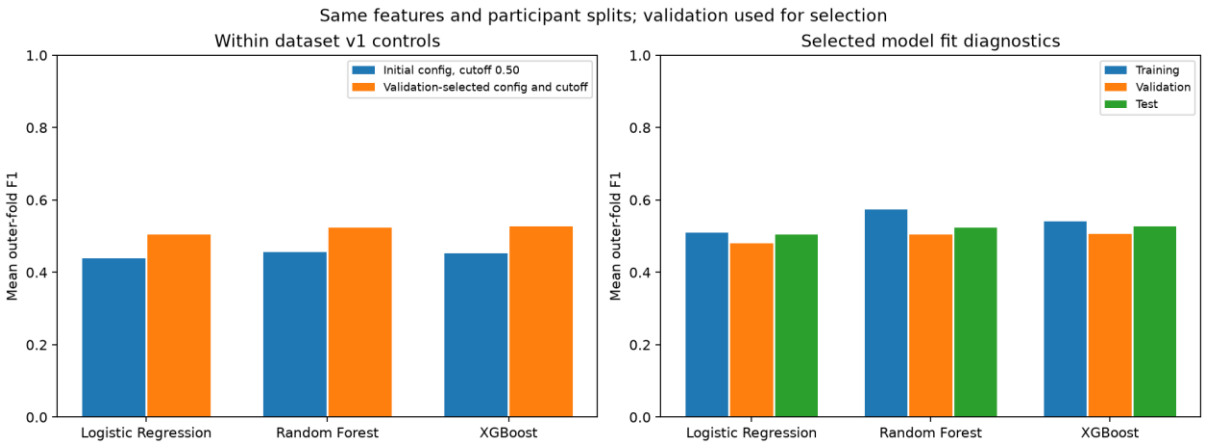
The prepared dataset changes session handling, participant eligibility, features and the proxy definition. Therefore, we *retrained* the models rather than applying the old fitted models to the new table. We also included **XGBoost**. To assess refinement fairly, we compare initial and selected settings within this new experiment. A direct difference from the older scores would mix changes in data preparation with changes in modeling.

The initial learned-model controls use the first configuration listed in *Section 4* and a decision threshold of 0.50. The following values are mean test metrics across the three outer folds. The rule remains unchanged.

Model	Initial F1	Selected F1	Initial AP	Selected AP
Rule	0.4496	0.4496	0.2779	0.2779
Logistic Regression	0.4379	0.5039	0.4881	0.4881
Random Forest	0.4565	0.5242	0.5389	0.5405
XGBoost	0.4531	0.5269	0.5357	0.5420

In the new controls, **Logistic Regression** and **XGBoost** each detect approximately one third of positive cases at threshold 0.50. **Random Forest** has recall 0.7332 but precision 0.3314 and about 7.04 false-positive predictions per evaluated participant-local day. This supports examining the decision threshold alongside the model configuration. *Figure 1* compares the initial and selected settings and shows training, validation and test F1.

Figure 1. Initial versus selected settings and fit diagnostics.



3. Refinement Techniques

We used *two related refinements*: comparing a new algorithm, XGBoost, and selecting a small set of configurations and decision thresholds using validation participants. Logistic Regression provides a **linear baseline**. Random Forest and XGBoost can represent **nonlinear relationships** between usage features. Their greater flexibility is useful to investigate, but does not guarantee better performance on new users.

All learned models use *median imputation* and *missing-value indicators* fitted only on training participants. Logistic Regression additionally applies *StandardScaler*. Random Forest uses *balanced class weights* calculated from training labels. Logistic Regression is *unweighted*, and XGBoost uses *scale_pos_weight=1*. No resampling was performed. Both tree-model implementations accept sample weights, which prepares the code for later personalization experiments; *it does not mean that personalization has already been evaluated.*

The **fixed rule** uses only *current-window inputs*. It predicts positive if the longest within-window use reaches 20 minutes, if at least six sessions start, or if the window starts between 00:00 and 05:00 and contains at least five screen-active minutes. Its score is the fraction of matched conditions, not a probability. We did not use the current retrospective lab_any annotation as a predictor, because its long-session component can depend on when a session ends after the prediction time.

4. Hyperparameter Tuning

The candidate list was fixed before examining outer-test results. We used *two Logistic Regression configurations*, *two Random Forest configurations* and *three XGBoost configurations*: seven fits per outer fold and 21 fits in total.

Model	Configurations tested
Logistic Regression	C = 1.0 or 0.1
Random Forest	(max_depth, min_samples_leaf) = (8, 5) or (12, 10)
XGBoost	(max_depth, n_estimators) = (3, 150), (5, 150) or (3, 300)

Logistic Regression uses *max_iter=1000* and *random_state=42*.

Random Forest uses *100 trees, class_weight="balanced", random_state=42* and *n_jobs=1*.

XGBoost uses *learning_rate=0.05*, *min_child_weight=5*, *subsample=0.8*, *colsample_bytree=1.0, reg_lambda=1.0, scale_pos_weight=1.0, objective="binary:logistic", eval_metric="logloss", tree_method="hist", random_state=42* and *n_jobs=1*.

These modest candidates allow a controlled comparison without a large search.

For each configuration, we evaluated *decision thresholds* from 0.05 to 0.95 in steps of 0.05. We selected the configuration and threshold with the *highest validation F1*. Threshold ties prefer the higher threshold, and configuration ties prefer the first listed configuration. Test labels were not used for selection. There was no early stopping and no refitting on the combined training and validation sets after selection.

Fold	Model	Selected configuration	Threshold
1	Logistic Regression	C=1.0	0.25
1	Random Forest	depth=12, leaf=10	0.70
1	XGBoost	depth=5, trees=150	0.25
2	Logistic Regression	C=1.0	0.25
2	Random Forest	depth=12, leaf=10	0.65
2	XGBoost	depth=3, trees=300	0.25
3	Logistic Regression	C=1.0	0.25
3	Random Forest	depth=12, leaf=10	0.75
3	XGBoost	depth=5, trees=150	0.30

Logistic Regression selected *C=1.0* in all folds, so its F1 improvement comes from the decision threshold. **Random Forest** selected *depth 12* and *minimum leaf size 10* in every fold. **XGBoost** selected *depth 5 with 150 trees in folds 1 and 3*, and *depth 3 with 300 trees in fold 2*. Since configuration and threshold were selected jointly, the total improvement cannot be attributed to hyperparameters alone. *These fold-specific choices are not a single final deployment configuration.*

5. Cross-Validation

We retained the *participant-based approach* from the previous *threshold experiment*: *three-fold GroupKFold for outer testing*, with *GroupShuffleSplit* reserving 20% of each development group's participants for validation (random_state=42).

Each fold contains *176 training, 44 validation* and *110 test participants*. No participant belongs to more than one of these sets within a fold. Every eligible participant appears in exactly one outer-test fold.

Fold	Training rows	Validation rows	Test rows
1	191,668	49,823	120,764
2	194,517	47,011	120,727
3	190,767	50,724	120,764

The inner step is one grouped validation holdout, not inner k-fold cross-validation. The main changes are *the prepared cohort* and *configuration search*; we did not switch to a random row split. Keeping users separate is necessary because windows from the same person are related. The public cohort has already been explored, so these tests estimate performance on held-out users within this study rather than on a new external population.

6. Feature Selection

All learned models receive the same seven inputs: *screen_s, sessions_started, longest_run_s, app_switches, prev_session_s, hour and dow*. They describe **current-window screen duration, new session starts, the longest continuous overlap, application changes, previous session duration** and **local temporal context**. In this source, *prev_session_s* refers to a session ended at or before window start and is capped at three hours; hour and dow also refer to window start.

Feature selection was *manual*. Notifications were excluded because 63 participants have a different notification-event variant. User identity, quality flags, target columns, current labels, raw repeated unlock counts and retrospectively cleaned screen duration were *excluded* from predictors. Personal-history deviations and hist_days were *left for later controlled history experiments*. We did not run automatic feature selection, an ablation study or SHAP analysis, so we cannot attribute a measured gain to individual features.

Test Submission

1. Overview

This phase evaluates the frozen selected models on the outer-test participants and checks that saved models reproduce their predictions. *Evaluation is offline. The completed work does not include an Android deployment or a submission to an external prediction server.*

2. Data Preparation for Testing

The **preparation pipeline** is stored in *src/data/mpu_pipeline.py*. The package contains *443,781 windows from 342 participants*, of which *372,349 satisfy the source heartbeat-validity rule* and *366,362 have valid adjacent targets*. We applied the supplied participant-eligibility flag: screen events, at least seven valid-window-equivalent days, at least seven active screen days and a mean daily session-start count of at least one. This excludes 12 participants and 4,107 target-bearing rows. The **final experiment** contains **330 participants** and **362,255 window pairs**, with **41,968 positive targets** (11.59%).

Before filtering, we checked *window order, unique participant/time keys, 30-minute spacing, validity flags* and *all four next-window target shifts*. Target alignment had zero mismatches. We also checked the inventory and summary counts. As a separate sample check, regenerating the first five participants reproduced *6,464 rows*, and cutting future events from 70 selected windows produced zero feature mismatches. *These sampled checks do not prove that every possible recording problem is absent.*

The *target y_next_any* combines three supplied proxy components in the immediately following valid window: **a valid intersecting session** lasting at least 20 minutes, night-time clean screen use of at least five minutes between 00:00 and 05:00, or at least six session starts. *These source thresholds were held fixed in this experiment; they were not learned from our fitting subsets or optimized on outer-test scores.*

There are important limits to this target. The *long-session component* uses the complete duration of a session intersecting the next window. It can therefore depend on events after that window ends. Sessions longer than three hours are excluded retrospectively from these labels. The current result is prediction of a retrospective proxy annotation associated with the next window, rather than a strictly bounded next-30-minute outcome. *This must be addressed before chronological personalization tests.*

The source clock uses the participant timezone plus a fixed one-hour adjustment. Its hour/day fields are internally consistent, but full timezone and daylight-saving histories have not been validated. A *valid window requires* at least one designated heartbeat event, which does not guarantee continuous observation. *Participant eligibility* also uses retrospective recording summaries. These assumptions limit conclusions about live collection. Test rows are transformed using training-only preprocessing, without fitting new statistics on the test participants.

3. Model Application

Each trained classifier produces a positive-class score. We apply the *configuration* and *threshold* selected on *validation users* to its corresponding test users. Nine selected models were saved with joblib: three algorithms across three folds. Their preprocessing and selected thresholds were preserved. *After reloading*, every model reproduced all of its test scores and decisions exactly. This verifies Python save/load consistency, not mobile compatibility.

4. Test Metrics

We report **precision, recall, F1, average precision (AP), ROC-AUC** and **false-positive predictions** per evaluated participant-local day. *AP* is calculated with *sklearn.metrics.average_precision_score*; it is not trapezoidal PR integration. The following values are unweighted means across the three outer folds. They are not averages across individual participants.

Model	Precision	Recall	F1	AP	ROC-AUC	FP/day
Rule	0.5201	0.3960	0.4496	0.2779	0.6742	1.740
Logistic Regression	0.4884	0.5204	0.5039	0.4881	0.8354	2.595
Random Forest	0.5191	0.5334	0.5242	0.5405	0.8488	2.386
XGBoost	0.5204	0.5355	0.5269	0.5420	0.8503	2.350

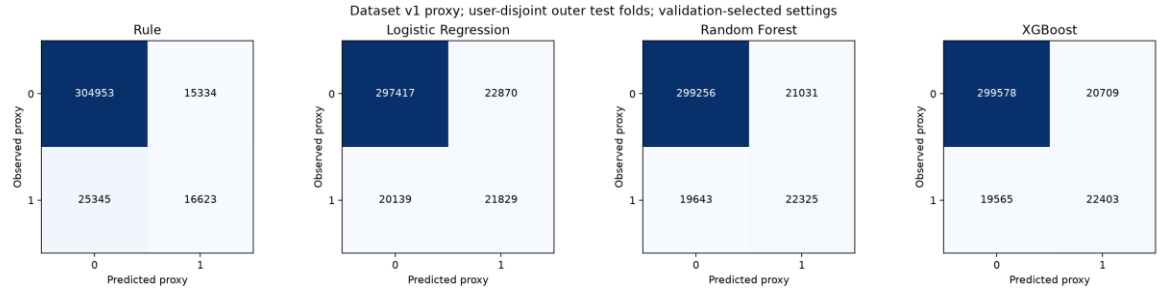
The *FP/day denominator counts* distinct evaluated participants and local prediction dates, including partial dates. It describes incorrect model predictions, not the number of notifications actually shown. Notification limits and quiet hours have not been applied in this offline study.

Selected model	Training F1	Validation F1	Test F1
Logistic Regression	0.5098	0.4794	0.5039
Random Forest	0.5737	0.5040	0.5242
XGBoost	0.5413	0.5058	0.5269

Random Forest has the largest mean training–test F1 gap, about 0.0495, compared with 0.0144 for *XGBoost* and 0.0059 for *Logistic Regression*. This suggests that RF fits the training participants more closely, although the different user distributions also affect these differences. *Validation scores were used for selection and are not an independent performance estimate.*

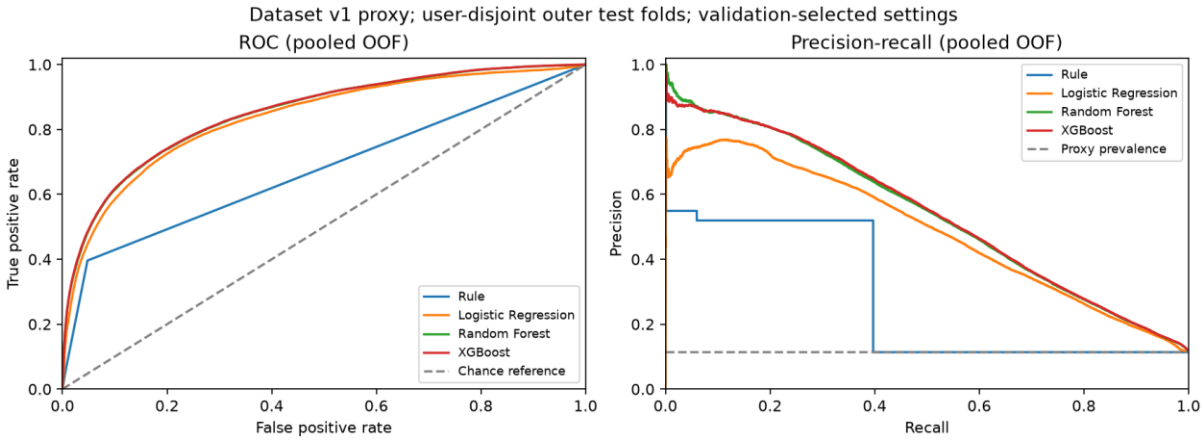
XGBoost has the highest mean test F1 (0.5269) and AP (0.5420), but its advantage over *RF* is only about 0.0027 F1 and 0.0015 AP. *No confidence intervals or significance tests were calculated, so this is not sufficient to establish a clear winner*. The selected RF settings reduce FP/day from 7.041 to 2.386, while selected LR increases it from 0.895 to 2.595 as recall improves. *Thus a higher F1 does not necessarily mean fewer false alerts.*

Figure 2. Confusion matrices pooled over outer-test predictions, using the selected settings.



Model	True negative	False positive	False negative	True positive
Rule	304953	15334	25345	16623
Logistic Regression	297417	22870	20139	21829
Random Forest	299256	21031	19643	22325
XGBoost	299578	20709	19565	22403

Figure 3. ROC and precision–recall curves pooled over outer-test predictions.



5. Model Deployment

The models have been saved and checked in Python, *but they have not been deployed on Android.* The project still needs matching feature calculations on the phone, a supported model format, device tests and the reminder/feedback interface. *The saved joblib files are research artifacts and should not be described as mobile-ready models.*

6. Code Implementation

The following examples use the same functions as the experiment. The first shows ***configuration and threshold selection with training and validation inputs only***.

The full implementation, including the fixed candidate list, is in *src/models/refinement_exploration.py*.

```
from src.models.refinement_exploration import CANDIDATES, select_candidate
from src.models.threshold_exploration import participant_splits
# Split by participant before fitting any preprocessing or classifier.
fold, train_idx, val_idx, test_idx = next(participant_splits(table))
training = table.iloc[train_idx]
validation = table.iloc[val_idx]
test = table.iloc[test_idx]
# The test frame is not passed to configuration/threshold selection.
initial, selected, search = select_candidate(
    training, validation, "XGBoost", CANDIDATES["XGBoost"])
model, threshold, parameters = selected
scores = model.predict_scores(test)
predictions = (scores >= threshold).astype(int)
```

The next example reloads the saved ***fold-1 XGBoost model*** and applies it to the recorded test users. The fitted preprocessing and selected cutoff are already stored in the model.

```
import json
import joblib
from pathlib import Path
from src.models.dataset_v1 import load_v1
# Validate the source data and provenance before selecting held-out rows.
table, coverage, manifest = load_v1(
    "data/processed/dataset_v1", "src/data/mpu_pipeline.py")
local = json.loads(Path(
    "data/processed/model_refinement_dataset_v1/local_details.json"
).read_text())
test_users = local["splits"][0]["test"]
test = table.loc[table.participant_id.isin(test_users)]
model = joblib.load(
    "models/refinement_dataset_v1/fold_1/xgboost.joblib")
scores = model.predict_scores(test)
predictions = model.predict(test)  # Uses its saved validation cutoff.
```

The full experiment runs with ***python -m src.models.refinement_exploration*** from the repository root.

Tests run with ***python -m pytest -q --tb=short***. The verified suite contains 86 passing tests covering model input checks, sample weights, target alignment, participant separation, selection behaviour and save/load consistency.

The recorded environment is *Python 3.11.15, NumPy 2.4.6, pandas 3.1.0, scikit-learn 1.9.1, XGBoost 3.2.0, matplotlib 3.11.2* and *joblib 1.6.0*.

Conclusion

We completed *a reproducible general-model comparison on the prepared dataset* and *improved the initial F1 values* through validation-based configuration and threshold selection. *XGBoost* achieved mean test F1 0.5269 and AP 0.5420, with only a small advantage over Random Forest. The models still miss a substantial share of positive cases and generate false positives, so the results support further evaluation rather than a claim of deployment readiness or recognition of genuinely unwanted use.

Personalization remains planned work for the remaining project period. We intend to compare a general model, a personal-only model and a model retrained with higher weight on the user's earlier records, using 1, 3, 7 and 14 recorded days where sufficient history exists.

Before those experiments, the label timing must be resolved so that training labels do not use information from the future test period. These experiments can first use the existing public logs. Later, consent-based phone recordings and user feedback will be needed to assess practical use. *No personalization or user-feedback performance is claimed in this report.*

References

1. **Group 8. Concept Note and Implementation Plan; Literature, Data and Technology Review; Data Preparation and Model Exploration.** Project documents in documents/.

2. **Dataset v1 and Mobile Phone Use preparation pip**eline, src/data/mpu_pipeline.py.

3. **Project implementation and results**: src/models/dataset_v1.py, tree_models.py, refinement_exploration.py and threshold_exploration.py; reports/model_refinement_dataset_v1/results.json.

4. **XGBoost developers. Python API Reference.** https://xgboost.readthedocs.io/en/stable/python/python_api.html

5. **scikit-learn developers. GroupKFold and average_precision_score documentation.** https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.GroupKFold.html ; https://scikit-learn.org/stable/modules/generated/sklearn.metrics.average_precision_score.html